

Introduction to Programming: Introduction, variables, types and statements

Reading

- [PY4E chapter 1 \(https://www.py4e.com/html3/01-intro\)](https://www.py4e.com/html3/01-intro)
- [PY4E chapter 2 \(https://www.py4e.com/html3/02-variables\)](https://www.py4e.com/html3/02-variables)

Getting started ¶

Interactive shell

The easiest way to start working with Python is by using the interactive shell.

Log in to the server, triton, using your regular SU login:

```
ssh xxxx1234@triton.dsv.su.se
```

and run the `ipython3` command. You should then see something like this

```
robert@ros:~$ ipython3
Python 3.11.2 (main, Mar 13 2023, 12:18:29) [GCC 12.2.0]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.5.0 -- An enhanced Interactive Python. Type '?' for help.
```

```
In [1]:
```

You can now type in instructions for Python to execute.

What can we type?

Python understands several types of **"nouns"**, for instance:

Numbers

```
In [1]: -1
```

```
Out[1]: -1
```

```
In [2]: 3.2412
```

```
Out[2]: 3.2412
```

```
In [3]: 2.13e-3
```

```
Out[3]: 0.00213
```

Strings

```
In [4]: 'python is great'
```

```
Out[4]: 'python is great'
```

```
In [5]: "python is great"
```

```
Out[5]: 'python is great'
```

Lists

```
In [6]: [1, 'python', -234]
```

```
Out[6]: [1, 'python', -234]
```

There are many other kinds of objects Python knows about. In fact, you can define your own types of objects yourselves. We will be doing it later in the course.

Python also knows some **"verbs"**, for instance:

Mathematical operators

```
In [7]: 2 * 7
```

```
Out[7]: 14
```

```
In [8]: 8.1 / 3
```

```
Out[8]: 2.6999999999999997
```

```
In [9]: 4 % 3
```

```
Out[9]: 1
```

String operations

```
In [10]: 'Hello' + ' ' + 'world' + '!'
```

```
Out[10]: 'Hello world!'
```

```
In [11]: 'python'.upper()
```

```
Out[11]: 'PYTHON'
```

Logical operations

```
In [12]: 3 > 5 and 4 < 1
```

```
Out[12]: False
```

Other built-in functions

```
In [13]: len('Hello world!')
```

```
Out[13]: 12
```

Again, you will soon (today, in fact!) learn how to define and use your own functions.

Scripts

The interactive session is volatile. When you close it, everything you have typed in will be lost.

For this reason, we usually save our programs in files. Conventionally, Python scripts use `.py` extension.

Your first script

Create a new script by running the following command (`micro` is just a name of simple command-line editor):

```
micro hello.py
```

Next, type the following program:

```
print('Hello world!')
```

Save the file (`Ctrl-S`) and run your program by typing

```
%run test.py
```

from within IPython. Alternatively, you could run it from the regular Python interpreter:

```
python3 test.py
```

Printing vs. returning

In the interactive interpreter, the returned value is automatically printed on the screen:

```
In [2]: 2 + 2  
Out[2]: 4
```

However, if we put the same expression in a script and run it, nothing will be printed. You have ask for it explicitly by saying:

```
print(2 + 2)
```

Exercises and automatic code checker

Apart from the exercises in the PY4E book, we also have a set of our own language-themed exercises for this course. We strongly recommend that you solve all of them during the first month of this course, in order to master the basics of Python ahead of the (mandatory!) lab assignments during the second month:

- [Python exercises for linguistics \(https://people.dsv.su.se/~robe/lis020/exercises/\)](https://people.dsv.su.se/~robe/lis020/exercises/)

In this course we will be using a tool (written in Python, of course!), which can check whether your solutions to most of the exercises are correct, or help you find cases where your code does not work as expected.

Simply run the tool `test-lis020` in the directory where your solutions are stored. For instance, after solving only the [Returning the second argument exercise \(https://people.dsv.su.se/~robe/lis020/exercises/second.html\)](https://people.dsv.su.se/~robe/lis020/exercises/second.html), which according to the instructions should be saved in the file `simple.py`, the output should look something like this:

```
robe@triton:~/exercises-lis020$ test-lis020
Testing your LIS020 exercises in /home/dsv/robe/exercises-lis020...
Running tests.
  simple.second: OK
* simple.censor: no such function defined
* simple.words_overlap: no such function defined
* simple.longest_chain: no such function defined
* The following modules could not be loaded:
  database english experiments information mandarin morphology phonetics phonology rawtext signlanguage swedish syntax translation turkish
Please ensure that the corresponding .py files exist
```

As you progress further, you will see more "OK" and fewer error messages (beginning with an asterisk, *). Eventually you will reach the stage where everything is OK.

One more complication...

To make things a bit more complicated, you need to wrap your solutions into *functions*.

You will learn about functions later in the course. For now, it is sufficient to say that a function is basically a sequence of Python statements which has a name and which you can run by calling that name.

So we can re-write our `hello.py` program as a function as follows:

```
In [14]: def hello():  
         print('Hello world')
```

We can then call the function by saying:

```
In [15]: hello()  
  
Hello world
```

Arguments and values

Functions can also take arguments...

```
In [16]: def hello(x):  
         print('Hello ' + x + '!')  
  
         hello('python')  
  
Hello python!
```

... and return values:

```
In [17]: def multiply(x, y):  
         return x * y  
  
         multiply(2, 23)
```

```
Out[17]: 46
```

Again, if you call a function from a script, you have to use the `print()` function to see anything appear on the screen:

```
In [18]: print(multiply(2, 23))
```

```
46
```

Variables, types and statements

We will now dive deeper into three very fundamental concepts in Python, whose formal terms are *variables*, *types* and *statements*. Our goal today is that you will get an intuitive understanding of what those terms mean.

Types and objects

Python is a so-called *imperative programming language*, which means that it essentially consists of a bunch of instructions (*statements*) telling the computer to perform certain actions on a set of *objects*.

It may help you for now to imagine each object as a physical object, perhaps a little paper card with something written on it. One problem with this analogy is that you can just as well interpret a card with "17" written on it, as one with "seventeen". Python, on the other hand, makes a strict separation between different *types* of objects.

To make things less abstract, we will not take a tour of some common types in Python.

Booleans

There are two boolean values, called `True` and `False`. They are usually used to represent answers to yes/no questions, and we will return to them later in this lecture.

Integers

To create an integer, simply type a decimal number:

```
In [19]: 17
```

```
Out[19]: 17
```

Floats

A frequent source of confusion and trouble is that Python, like most programming languages, makes a distinction between integers (whole numbers) and so-called *floating-point values* (numbers with decimals). If you type a number with a decimal point in it, it becomes a value of type `float` :

```
In [20]: 17.0
```

```
Out[20]: 17.0
```

If you are not sure which type a certain value has, you can ask Python to tell you:

```
In [21]: type(17)
```

```
Out[21]: int
```

```
In [22]: type(17.0)
```

```
Out[22]: float
```

Strings

A string in Python is a sequence of characters. The important thing to keep in mind is that Python makes no further interpretation of the contents of a string. These strings are all "equal", even though a human easily recognizes them as belonging to the categories of "name", "number (whole)", "number (decimal)", "nothing" respectively:

```
In [23]: "John"
```

```
Out[23]: 'John'
```

```
In [24]: "17"
```

```
Out[24]: '17'
```

```
In [25]: "17.0"
```

```
Out[25]: '17.0'
```

```
In [26]: ""
```

```
Out[26]: ''
```

If you pay close attention, you may have noticed that while we enclosed the contents of each string in double quotes, Python replied with single quotes. Both may be used and work the same:

```
In [27]: '17'
```

```
Out[27]: '17'
```

The importance of getting types right

Now we have a whole zoo of objects representing 17 in different ways. What is the difference between them? We illustrate by trying to add some combinations

```
In [28]: 17 + 17
```

```
Out[28]: 34
```

Python knows how to add integers. A plus sign performs this operation. So far, no surprises.

```
In [29]: 17 + 17.0
```

```
Out[29]: 34.0
```

Integers can apparently be added to floats without problems. Note, however, that the result is no longer an integer but a float. This behavior is hard-coded into Python because the alternative (returning an integer) would risk losing decimals.

What about adding an integer to a string?

```
In [30]: 17 + "17"
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In [30], line 1  
----> 1 17 + "17"  
  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Python does not want to do this, and for good reasons. Look what happens if we try to add two strings:

```
In [31]: "17" + "17"
```

```
Out[31]: '1717'
```

Remember that Python does not try to interpret the contents of strings. They are just a bunch of characters. The plus sign means something else with strings: concatenation. No wonder it refused to guess what `17 + "17"` is. Could it be the integer 34? Or the string "1717"? Or even the string "34"?

Type conversions

Sometimes we have an object in one type, but need it in another type. We can use `int`, `float` and `str` to convert to these respective types. For instance:

```
In [32]: int("17") + 17
```

```
Out[32]: 34
```

```
In [33]: str(17) + str(17.0)
```

```
Out[33]: '1717.0'
```

```
In [34]: str(17 + 17.0)
```

```
Out[34]: '34.0'
```

Variables and statements

You can always create an object and use it directly, by typing a value in the correct format (e.g. 17, 17.0 or "17"). The real power of programming comes from the fact that you can work with values that you do not know at the time of writing the code.

A *variable* is a name given to an object, and can be used in place of the actual object. Again, if you think of objects as little paper cards with their values written on them, you can think of variables as other paper cards with one end of a wire attached to them. The other end of this wire can be attached to an object card.

This is in fact a rather accurate analogy: each variable can be attached to exactly one object, but the same object may have many different wires attached to it, leading to multiple variables.

Variables can be created through *assignment* operations:

```
In [35]: first_variable = 17
```

```
In [36]: second_variable = 1
```

```
In [37]: first_variable + second_variable
```

```
Out[37]: 18
```

When we told Python to add `first_variable` and `second_variable`, what it actually does is to first check which object `first_variable` is connected to (17, which is still "out there"), then check which object `second_variable` is connected to (1), then it adds those two objects. The result is the integer 18, as expected.

Assignment is a syntactic structure in Python, and has a very specific meaning:

1. First perform whatever is on the right hand side of the equals sign. The result of this is an object.
2. The left hand side of the equals sign is the name of a variable. If the variable does not already exist, create it.
3. Attach the variable to the object.

To be specific, we take two examples:

```
In [38]: first_variable = 17
```

The three steps outlined above are now, in this specific example:

1. The right hand side says `17`. If we just type a number without decimals, this creates an integer object with the value 17.
2. The variable is named `first_variable`. Since it exists already (from the previous example), we do not need to create it.
3. Attach `first_variable` to the `17` integer object we just created.

Let us take a somewhat more complex example, step by step:

```
In [39]: third_variable = first_variable + 2
```

Again, the three steps:

1. The right hand side says `first_variable + 2`. Python first needs to turn this expression into a single object. This requires three sub-steps:
 - Check which object the `first_variable` variable leads to. It turns out to be the `17` object from the previous example.
 - Create an integer object with the value `2`.
 - Perform addition of the two integer objects we have (`17` and `2`). The result should be an integer object with the value `19` (we check this soon).
2. The variable on the left hand side is `third_variable`. This is a new variable, so Python now creates it for us.
3. The variable `third_variable` is now attached to the object described by the right hand side, i.e. the integer `19`.

To check that this worked as expected, we do this:

```
In [40]: third_variable
```

```
Out[40]: 19
```

Success!

The description above may seem overly detailed, but it is important to understand how Python works. Make sure that you understand it, and ask if you are uncertain.

Assignment is one type of *statement*, arguably one of the most important. There are many other statements in Python, such as if/elif/else (lecture 3), for and while loops (lecture 6).

Variable names

We named our variables so far `first_variable` , `second_variable` and `third_variable` . There are a few things to mention about naming variables. First, formal requirements:

- Variable names may not begin with a digit, but may contain digits later (e.g. `variable1` but not `1st_variable`).
- Variable names may contain upper- and lower-case letters, and are case sensitive (i.e. `variable` and `Variable` are distinct names).
- Variable names may not be one of Python's *reserved words*.

Second, but every bit as important, a code of conduct:

- Select informative variable names.
- Be consistent in creating variable names.
- Do not re-use variable names for other purposes.

If you break formal requirements, Python will complain. For instance:

```
In [41]: try = 17
```

```
Cell In [41], line 1
```

```
try = 17
    ^
```

```
SyntaxError: expected ':'
```

This is invalid because `try` is a reserved word (we will return to it in a later lecture), and thus can not be used as a variable name.

If you break the code of conduct, Python will not punish you, but you sooner or later your sins will come back to haunt you.

```
In [42]: var1 = 0.097
         var2 = 580
         x = var2 / var1
```

What is the purpose of the code above? Who knows. But if you take the same code and change some variable names...

```
In [43]: usd_per_sek = 0.097
         travel_cost_usd = 580
         travel_cost_sek = travel_cost_usd / usd_per_sek
```

Operators

We have already used some of Python's *operators*, namely `+` for addition (and for concatenating strings -- many operators have different meanings for different types) and `/` for division. There is a number of such operators, which you need to be familiar with:

- Arithmetic operators: `+` `-` `*` `/` `%` `//` `**`
- Logic operators: `not` `and` `or`
- Relational operators: `<` `>` `<=` `>=` `==` `!=`
- Bitwise operators: `<<` `>>` `|` `^` `&` (will not be used in this course, but note that e.g. `&` generally does *not* mean the same as `and`)

Let us say that we want to compute the answer to a yes/no question: "is this language endangered?" Assuming that the variables below are defined, we could code this in the following way:

```
In [44]: n_speakers = 400                                # Assignment
         proportion_bilinguals = 0.05                     # Assignment

         few_speakers = n_speakers < 10000               # Assignment + relational operator
         many_bilinguals = proportion_bilinguals > 0.5   # Assignment + relational operator
         endangered = few_speakers and many_bilinguals  # Assignment + logical operator
```

We can also combine several operators into a single *expression*:

```
In [45]: endangered = (n_speakers < 10000) and (proportion_bilinguals > 0.5)
```

Apart from the fact that fewer variables will be created, this single line of code does exactly the same thing as the three lines above, namely:

1. use the relational operator `<` (less than) to check whether the object that `n_speakers` refers to is less than 10000. The result of this is `True`.
2. use the relational operator `>` (greater than) to check whether the object that `proportion_bilinguals` refers to is greater than 0.5. The result of this is `False`.
3. use the logical operator `and` to check whether both answers were `True`. Since the second one was in fact `False`, the final answer is `False`.
4. make the variable `endangered` refer to the final answer, i.e. `False`.

From a linguistics perspective, this expression can be described with a context-free grammar.

Exercise: English vs Python

Draw a phrase-structure tree of the following English sentence: "endangered means below 10000 speakers or above 50% bilinguals".

Internally, Python stores the code line above in a data structure which is equivalent to the English phrase structure tree. The difference is that unlike English, Python's has been designed to be unambiguous. In English, we have ambiguity ("the man saw the elephant with a telescope") and we need some intelligence to disambiguate. Unfortunately, this reliance on intelligence tends to become a problem when dealing with the grammar of a programming language. A human will likely expect the code below to compute the area excluding margins as 81 (= 9*9).

```
In [46]: width = 10
         height = 10
         margin = 1
         area_without_margin = width-margin * height-margin

         area_without_margin
```

```
Out[46]: -1
```

Apparently, Python does not agree! Why? Because we have three arithmetic operations here, two subtractions and a multiplication. In order to make the grammar unambiguous, Python needs to decide which one to perform first. In this case, the relevant rules are:

- multiplication goes before subtraction.
- multiple operations of the same kind are performed from left to right.

In other words, the order is this:

1. multiply (`margin * height`), result is 10
2. leftmost subtraction (`width - 10`), result is 0
3. rightmost subtraction (`0 - margin`), result is -1

Python has a well-defined *operator precedence* that can be found here: <https://docs.python.org/3/reference/expressions.html#operator-precedence> (<https://docs.python.org/3/reference/expressions.html#operator-precedence>).

You do not need to memorize this, but be aware that it exists. Also be aware that when in doubt, you can always add parentheses!

```
In [47]: area_without_margin = (width-margin) * (height-margin)
         area_without_margin
```

```
Out[47]: 81
```

There we go. Remember: better too many parentheses than too few!

Bugs

You will inevitably make mistakes.

Some misakes cause your program to crash. Those are the *good* ones.

Other mistakes cause your program to act normal, except that everything (or worse: *some*) of what comes out of it is nonsense.

Debugging is an ancient art form that you will need to become intimately familiar with. Regarding mistakes that cause your program to crash immediately, typically due to syntactic errors in your Python code, the most important thing is to carefully read what Python complains about. Let us try:

```
In [48]: area_without_margin = (width-margin) * (heighth-margin)

-----
NameError                                Traceback (most recent call last)
Cell In [48], line 1
----> 1 area_without_margin = (width-margin) * (heighth-margin)

NameError: name 'heighth' is not defined
```

Do not underestimate simple misspellings. In this case, we wrote "heighth" instead of "height".

Another common problem:

```
In [49]: name = "John
         surname = "Doe"
         full_name = name + " " + surname

Cell In [49], line 1
      name = "John
            ^
SyntaxError: unterminated string literal (detected at line 1)
```

EOL means End of Line, Python was expecting to find a quotation mark `"` ending the string `"John"`. Instead, the line abruptly ended. Note that the error message does not tell you "hey, you forgot a quotation mark". Frequently you will have to figure out yourself what the underlying cause of the error is.

Now to the truly evil bugs, syntactically correct programs that misbehave in other ways. For instance, consider if we had made the following mistake:

```
In [50]: area_without_margin = (width+margin) * (height+margin)
```

Python has no way of knowing you meant to use a minus sign instead of a plus sign. You get an answer, and the answer is wrong. In this case it's not even very far off. That only makes it more difficult to detect the error.

A number of good habits can help you to avoid this subtle logical errors:

- Test your code systematically. This is actually what the tool `test-lis020` does: it calls your solutions with a number of specific values of the input variables, and verifies that your code computes the expected value.
- Keep your code as simple as possible. More code means more opportunities for mistakes.
- Avoid redundancy. Redundancy means more code, and again more opportunities for mistakes.
- Document your code well and choose good variable names. If the intention behind the code is spelled out, it is easier to spot later where the code deviates from this intention.

Automated testing is a big thing in software design, aimed mostly at large projects. The low-budget variant is to simply run your functions in the Python interpreter with a few example values. If you see anything unexpected, investigate.

Later in the course we will look at Python's built-in functionality for automatic testing.